



DocVault

Project Report

An offline, encrypted document vault for Android.

Purpose, requirements, architecture, implementation, engineering decisions, verification and outcome.

Field	Value
Application	DocVault
Application ID	com.docvault.app
Version	1.6.2 (build 10)
Platform	Android 7.0+ (API 24 to 36)
Framework	Flutter 3.47.0 (stable) / Dart 3.13.0
Status	Feature-complete for the offline scope; not yet published to Google Play
Distribution	Signed APK from a download link; AAB prepared for a future Play listing
Author	Gopi Patel
Report generated	23 September 2026

Figures in this report were measured from the source tree and build output at version 1.6.2, not estimated. Screenshots are renders of the shipping interface code. Functionality that is incomplete or unverified is identified as such in sections 14 and 19.

Contents

#	Section	Covers
1	Project overview	What the application is and who it is for
2	Objectives	What the project set out to achieve
3	Problem and requirements	The requirement, and what it ruled out
4	Solution overview	The shape of the answer
5	Key features	What the application actually does
6	The application in use	Eight annotated screens
7	System architecture	Layering, state, and the gate
8	Technology stack	Toolchain and every dependency, with reasons
9	Core workflows	Unlock, import, read, share
10	Engineering decisions	Problem, decision, implementation, result
11	AI-assisted development	How the work was actually produced
12	Implementation details	Crypto, data model, platform channel
13	Challenges and solutions	Every defect and build failure, diagnosed
14	Testing and verification	What is covered, and what is not
15	Security and data integrity	Threat boundaries and failure handling
16	Deployment and usage	Build pipeline, signing, client install
17	Role and contribution	Who did what
18	Results and metrics	Measured outcomes
19	Limitations	Known gaps, stated plainly
20	Future improvements	Grounded in the existing seams
21	Conclusion	Where the project stands
22	Master prompt	A specification that rebuilds the project

1 Project overview

DocVault is an Android application for storing personal and business documents on a phone in encrypted form. Documents are imported from the device, sealed with AES-256-GCM, and thereafter readable only through the application after a PIN or fingerprint unlock. Nothing is uploaded: there is no server, no account, and no network permission in the shipped manifest.

The intended user is someone who keeps identity documents, licences, agreements, bills and statements as photographs and PDFs on their phone, where they currently sit unprotected in a gallery or a downloads folder alongside everything else. The application is designed to be handed to such a person as a download link and used without instruction.

Aspect	Position
--------	----------

Aspect	Position
Primary user	A non-technical owner of documents - an individual, or a small business keeping its paperwork on a phone
Core promise	Documents stay readable to the owner and unreadable to anyone else who reaches the device storage
Operating model	Entirely on-device. No account, no backend, no network permission
Delivery	A signed APK installed from a link, with a plain-language install guide
Current state	Ten internal releases; feature-complete for the offline scope; not published to a store

2 Objectives

- Store arbitrary document files on an Android device so that the bytes on disk are useless without a secret only the owner holds.
- Keep the whole system offline, so that using it involves no account, no upload, and no third party who could be compelled or breached.
- Make the protected documents genuinely usable: viewable in the app, printable, shareable and exportable, rather than locked away in a form that drives the user back to unprotected copies.
- Produce an application a non-technical person can install from a link and operate without being taught.
- Ship a repeatable, verifiable release process - signed artefacts, version control of builds, and checksums - rather than an APK built by hand.
- Leave the architecture open to later cloud backup, accounts and multi-device sync without a rewrite, while shipping none of them prematurely.
- Be accurate with the user about what is protected and what is not, including where the protection ends.

3 Problem and requirements

3.1 The problem

Important documents accumulate on a phone as ordinary files. A photograph of an identity card sits in the camera roll; a tax statement sits in Downloads. Anyone who picks up an unlocked phone, and any application granted storage access, can read all of them. The usual alternatives each carry a cost: a cloud drive moves the problem to an account that can be compromised or locked out, and a password-protected archive is unusable on a phone for day-to-day access.

The requirement was therefore an application that protects documents at rest on the device itself, without introducing a remote party, and without making the documents so awkward to reach that the user keeps a second unprotected copy anyway.

3.2 Functional requirements

#	Requirement	Status
F1	Import any document file from the device and store it encrypted	Complete
F2	Gate all access behind a PIN, with optional biometric unlock	Complete

#	Requirement	Status
F3	Preview stored documents in-app without writing plaintext to shared storage	Complete for images, PDF and text; refused with an explanation for Office formats
F4	Organise documents by category, including categories the user creates	Complete
F5	Search and sort the document list	Complete
F6	Apply a second lock to individual sensitive documents	Complete
F7	Keep separate document sets for separate businesses	Complete
F8	Share a document to WhatsApp directly, with a fallback to the system share sheet	Complete
F9	Print a document, and export a decrypted copy deliberately	Complete
F10	Provide a recovery route for a forgotten PIN	Complete, as a recovery key
F11	Show recent activity across the vault	Complete
F12	Cloud backup, accounts, multi-device sync	Not implemented; interfaces and schema columns in place

3.3 Non-functional requirements

- Confidentiality at rest: authenticated encryption over whole documents, with the key never stored in recoverable form.
- No network dependency: the application must function with the device permanently offline, and must not request the INTERNET permission.
- Android 7.0 and above, which is where Keystore-backed secure storage becomes available.
- Installable outside a store, since the first delivery is a direct download rather than a Play listing.
- Honest failure behaviour: an unreadable document, a wrong PIN or an absent recovery key must produce a clear statement rather than a blank screen or a false reassurance.

A requirement that had to be refused

The initial request included a "forgot password" e-mail that would send a reset link. This is not implementable without defeating the design: the PIN is what unwraps the encryption key, nothing stores the PIN, and no server holds anything to reset against. Any mechanism able to restore access without the PIN would equally give access to whoever controlled that mechanism. The requirement was met instead with a recovery key - a high-entropy secret, shown once, that wraps a second copy of the vault key - and the interface states the trade-off rather than implying a reset exists.

4 Solution overview

DocVault stores document bytes as AES-256-GCM ciphertext in app-private storage, and their metadata - title, category, size, dates - in a local SQLite database. A random 256-bit data key encrypts the documents. That key is never derived from the PIN and never written in the clear: PBKDF2-HMAC-SHA256 turns the PIN into a key-encryption key, and only the wrapped result is persisted, in Android Keystore-backed secure storage.

Because the data key is independent of the PIN, changing the PIN re-wraps one small blob instead of re-encrypting the vault, and a wrong PIN fails as a GCM authentication error rather than as a comparison against a stored hash. A stolen copy of the secure-storage blob is therefore useless on its own.

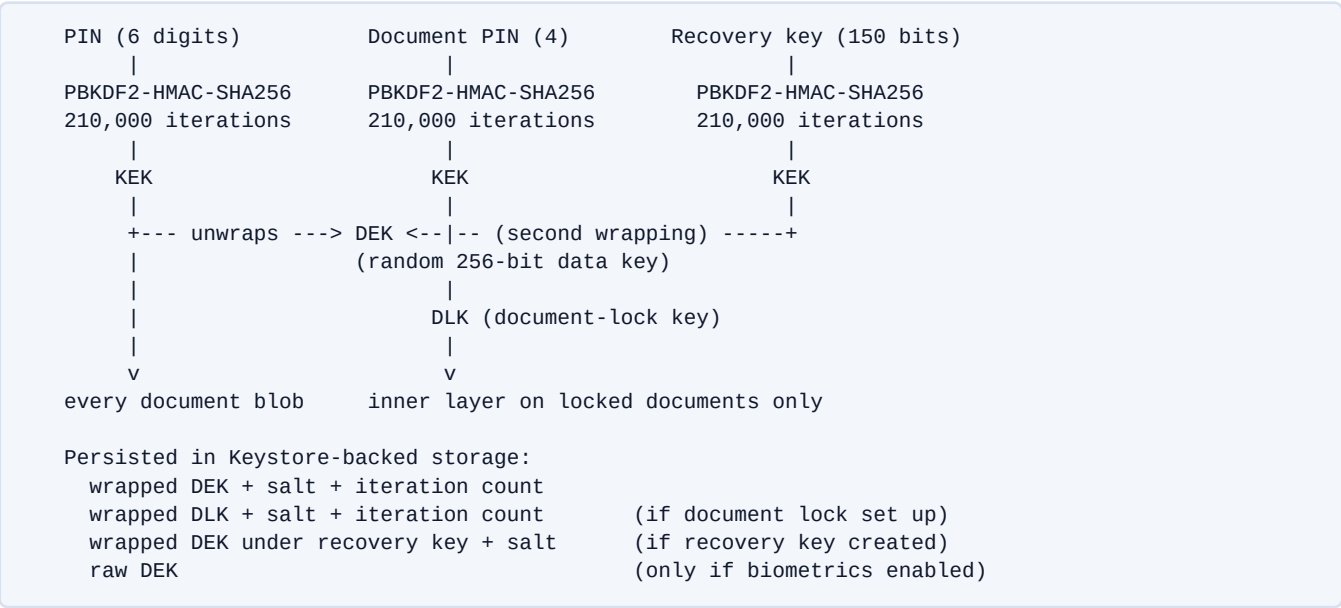


Figure 1. Key hierarchy. Only the wrapped forms are persisted; the data key exists in plaintext only in memory, and only while the vault is open.

Above that core sits an ordinary Flutter application: a lock screen, a company picker, and a four-tab shell containing the vault list, categories, activity and settings. The layers are kept apart deliberately - screens depend on a repository interface rather than on the storage implementation - so that an off-device backend can be added later without touching a widget.

5 Key features and capabilities

Area	Capability	Behaviour
Access	PIN unlock	Six digits on a drawn keypad rather than the system keyboard, so the PIN never passes through an IME that could learn or suggest it.
Access	Biometric unlock	Opt-in. Stores a second copy of the data key released after BiometricPrompt succeeds; removed the moment biometrics are turned off.
Access	Auto-lock	The key is dropped when the application leaves the foreground, so a warm app in the recents list is not a way past the lock screen.
Access	Recovery key	A 150-bit code in Crockford base32 groups, shown once, that wraps a second copy of the data key and can set a new PIN.
Storage	Encrypted import	The picked file is read, sealed and written to app-private storage. The blob is written before the metadata row, so a failure leaves a prunable orphan rather than a row pointing at nothing.
Storage	Per-document lock	An additional AES-GCM layer under a separate 4-digit PIN, inside the vault layer. Reading needs both keys.
Storage	Multiple companies	Independent document sets chosen after unlocking. Repositories are constructed per selection, so no query can forget to scope itself.

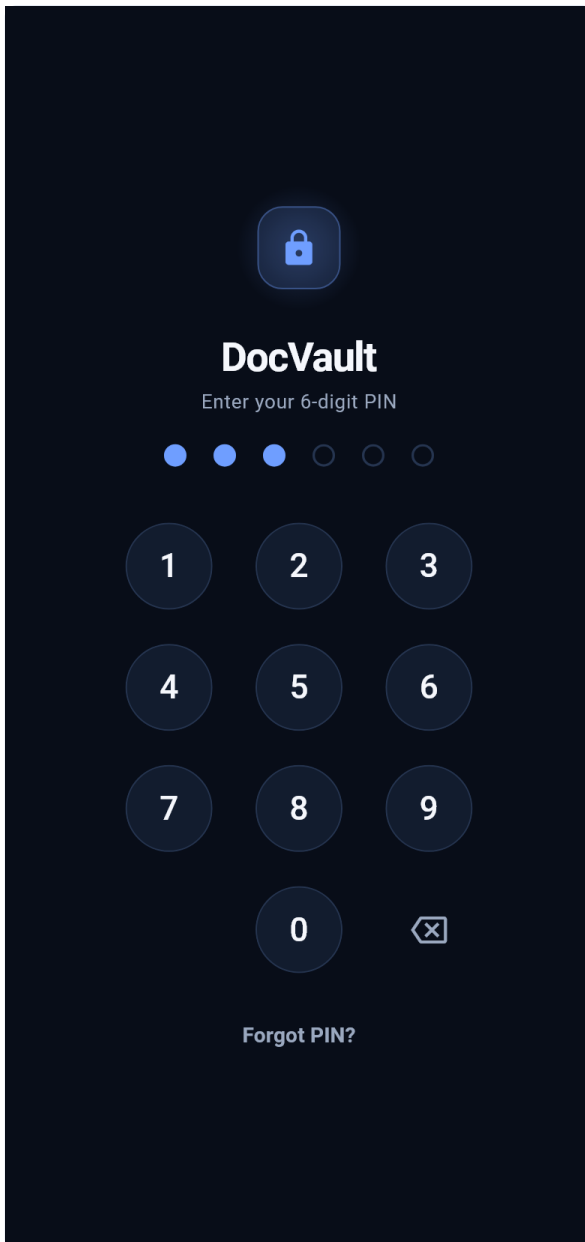
Area	Capability	Behaviour
Use	In-app preview	Images, PDF (including password-protected files, via bundled PDFium) and text render in the app. Office formats show a plain explanation instead of a blank box.
Use	Print	PDFs pass through untouched; images are wrapped in a single page. Formats that cannot be rendered are refused up front rather than printing blank.
Use	Share	Direct hand-off to WhatsApp when installed, with the system share sheet as fallback. One decrypted file is staged at a time in app-private cache.
Use	Export	An explicit "save a copy" through the system save dialog. The only route by which plaintext leaves deliberately.
Organise	Categories	Seven built-ins with fixed colours, plus user-created categories. "Other" is always last, being the catch-all and the fallback for a removed category.
Organise	Search and sort	Substring search across title, filename, category and notes; four sort orders.
Organise	Activity	Imports and edits, newest first, derived from the timestamps already on each document rather than from a separate log.
Manage	Settings	Biometrics, both PINs, recovery, vault statistics, backup status, version, and an irreversible erase.

Not in this release

Cloud backup, user accounts, multi-device sync, Google Drive and Dropbox integration, and a web dashboard are not implemented. They are represented in the code as feature flags in `lib/core/app_config.dart`, all currently false, with an abstract `SyncService` and reserved database columns behind them. The Settings screen reports backup status as "On this device only" rather than implying otherwise. Section 20 describes what each would take.

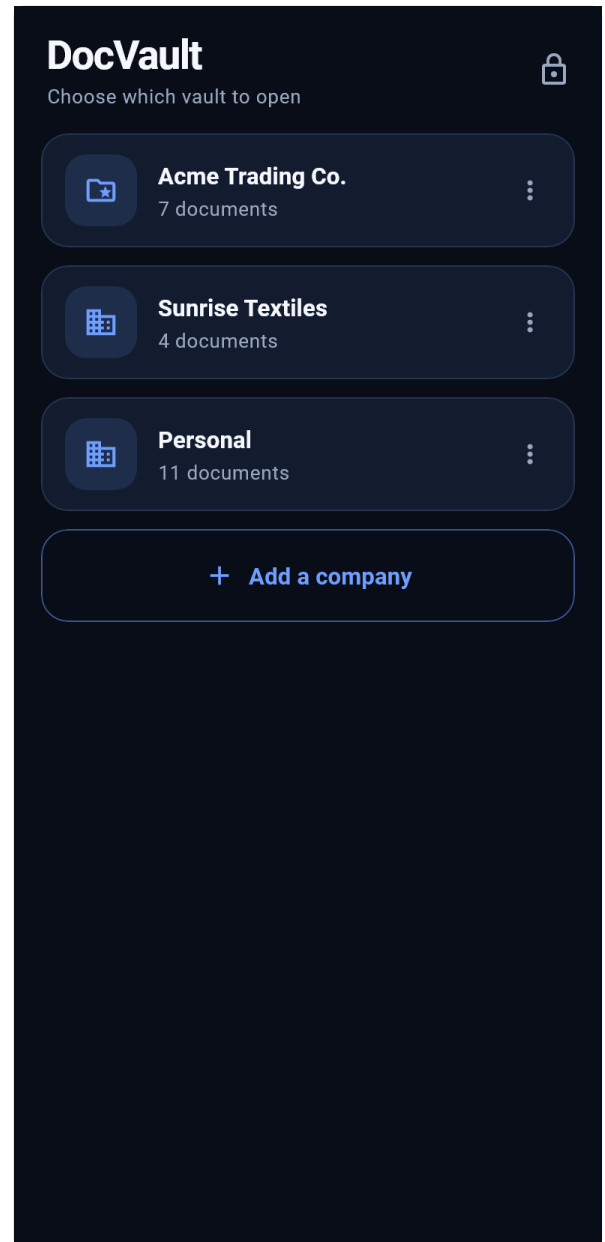
6 The application in use

Eight screens, in the order a user meets them. These are renders of the shipping widget tree rather than mock-ups: the same Dart code that runs on the phone, painted by the same Flutter engine, with the application theme and the real Roboto and Material icon fonts. They were rendered headlessly rather than captured from a handset; the note at the end of this section states exactly what that does and does not demonstrate. The vault shown contains invented sample data.



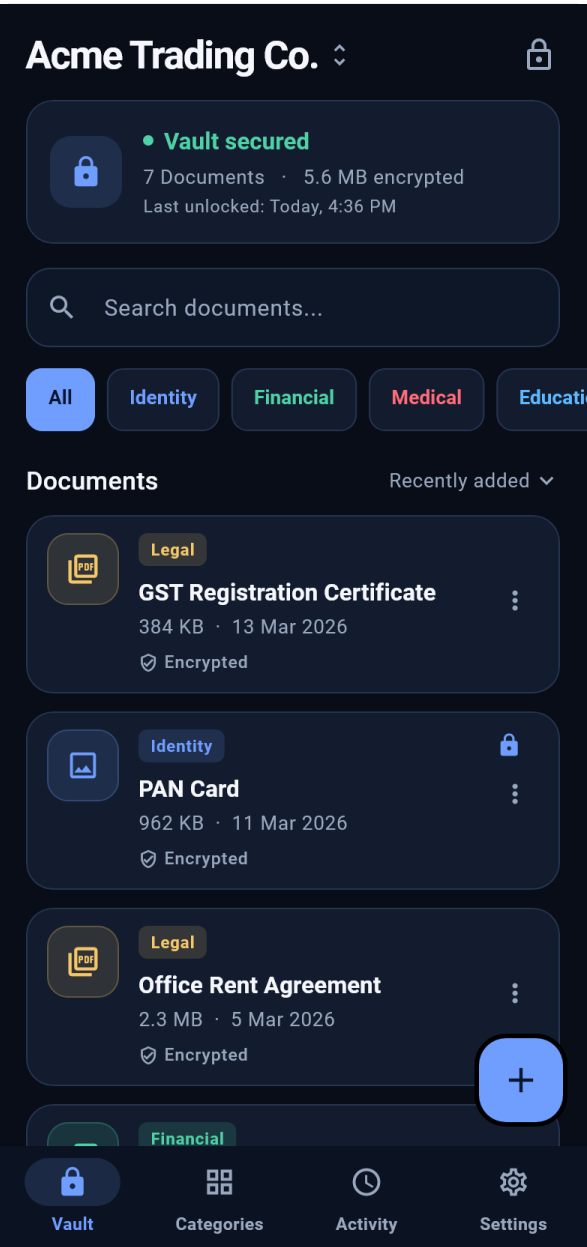
Lock screen

The application opens on this and nothing else. The keypad is drawn rather than using the system keyboard, so the PIN never passes through an IME. "Forgot PIN?" leads to the recovery key, or to a direct explanation when none was set up.



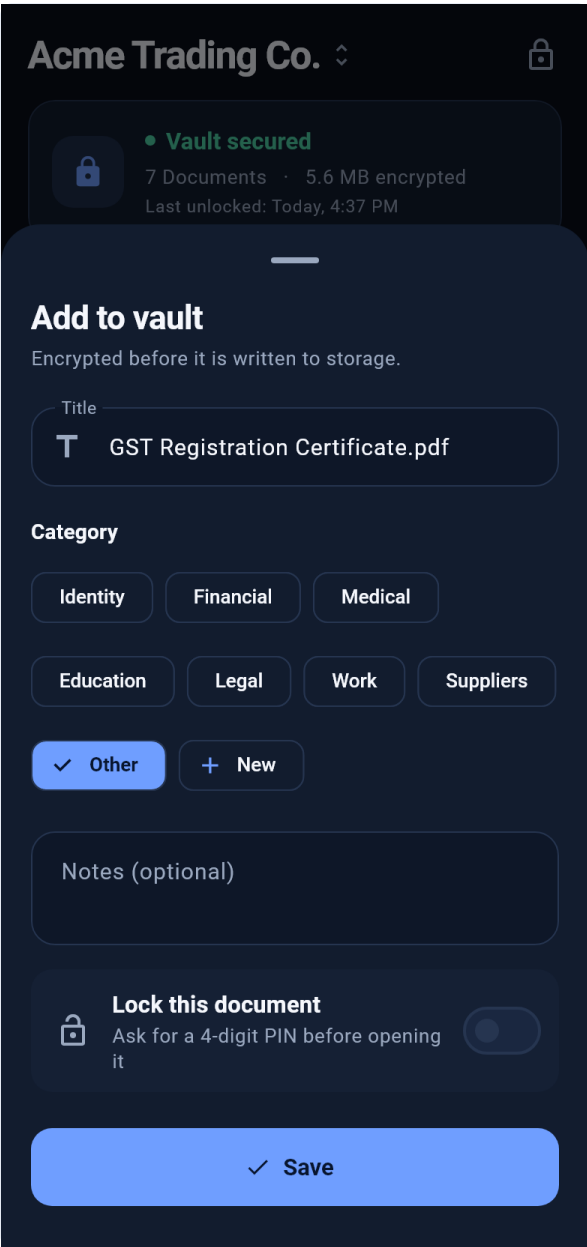
Company picker

Shown after the PIN, never before it: which vault to open is a choice, not a credential. Each card carries its document count. Companies partition the vault for organisation and share one encryption key, so this is not a security boundary.



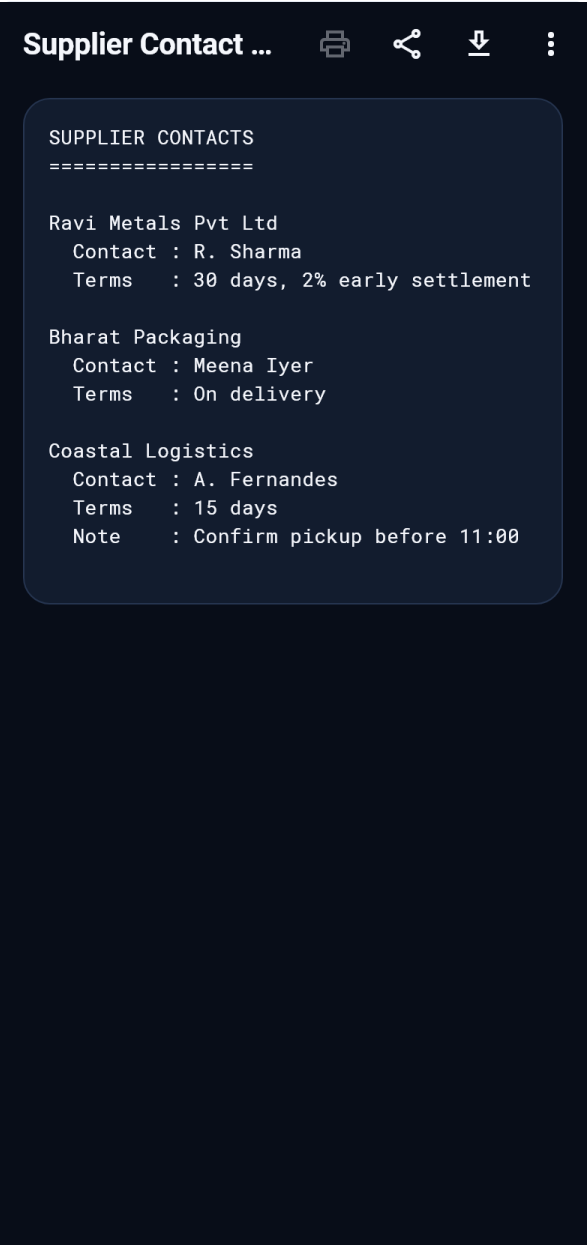
Vault - the document list

The status card answers "is my material safe" before "how much of it is there". Below it: search, the category chip row, a sort control, and the documents. The padlock badge on PAN Card marks a document behind the second 4-digit lock. Tapping the company name returns to the picker; the padlock at top right locks immediately.



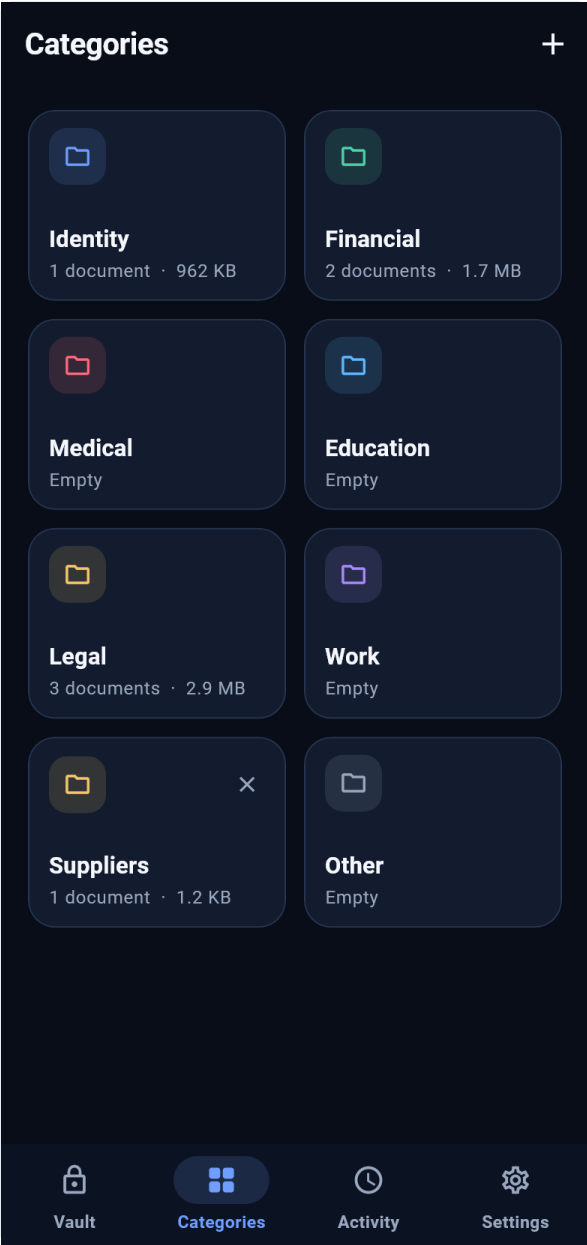
Add to vault

Raised over the vault once a file is picked. Title, category, optional notes and the per-document lock are all collected before any encryption starts, so backing out leaves nothing half-written. "New" adds a custom category. The subtitle states plainly that the file is encrypted before it reaches storage.



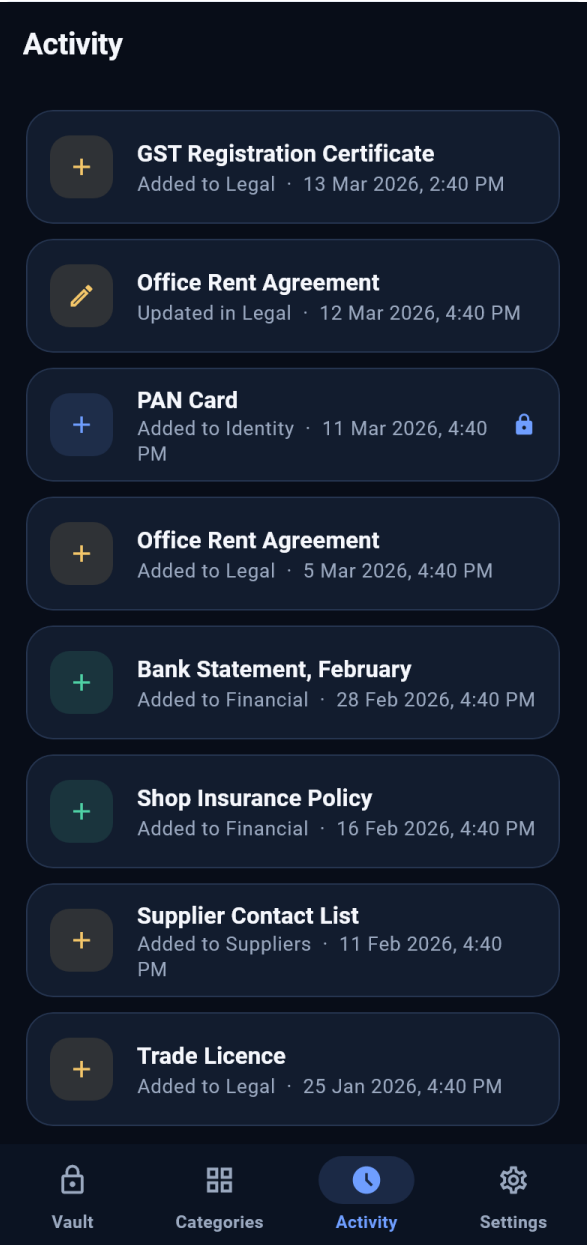
Document detail

The body is the document and nothing else. Print, share, save a copy and an overflow menu sit in the app bar; metadata lives behind the menu rather than filling the page. Shown here is the text renderer - images and PDFs get their own, PDFs through bundled PDFium so a password-protected file can still be opened.



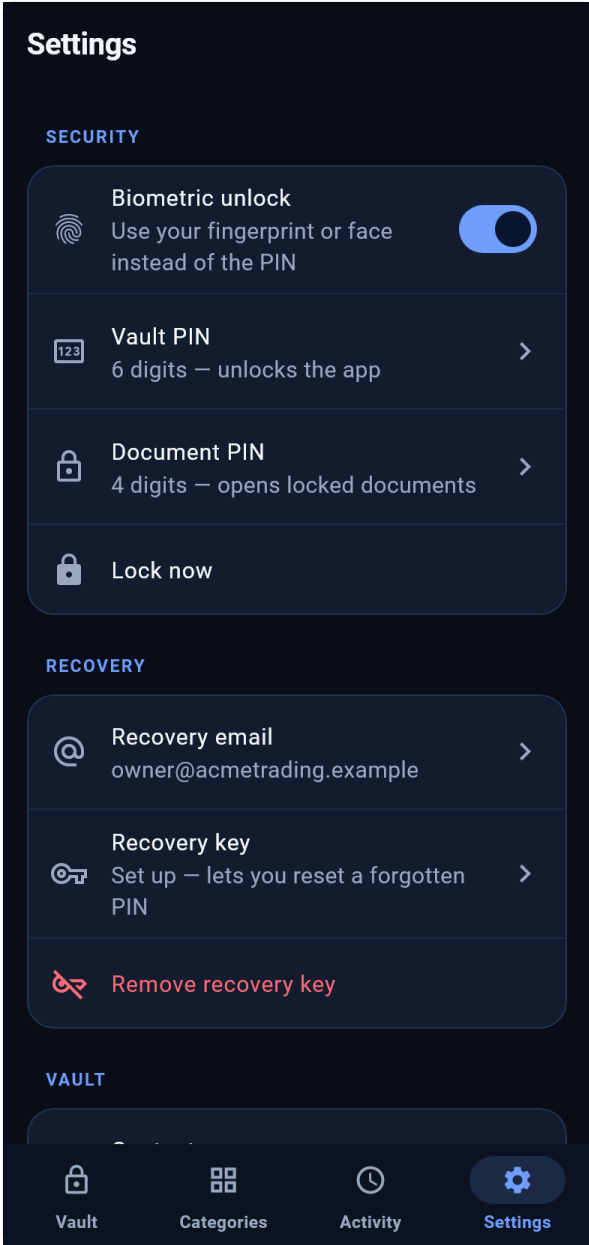
Categories

Every category with its document count and encrypted size, the empty ones included. Built-in categories take fixed colours so Financial is always green and Medical always red; custom ones hash into the same palette. Only a custom category carries a remove control.



Activity

Every import and edit, newest first. Derived from the timestamps already on each document rather than from a separate audit log, so it cannot drift out of agreement with the vault - and there is no second copy of the history to protect.



Settings

Security first: biometric unlock, the 6-digit vault PIN, the 4-digit document PIN and Lock now. Then recovery, which is the only route back from a forgotten PIN. Below the fold sit vault statistics, backup status - "On this device only" in this release - and the irreversible erase.

How these were produced, and what they do not prove

No device or emulator was attached to the machine that generated this report, so these are not adb screen captures. tool/screenshots/capture_test.dart drives the real screens through the real providers under the flutter_test binding and writes each frame to PNG. Only two edges are substituted, because neither exists off a device: the Android Keystore and the encrypted store. Everything above them - navigation, theme, layout, every widget - is what ships. These images therefore demonstrate the interface, not the encryption, the file picker, printing or the WhatsApp hand-off. Section 14 records how those were verified instead.

7 System architecture

The application is layered, and the layering is enforced by a single rule: `lib/providers.dart` is the only file where the layers meet. Screens read providers; providers construct repositories; repositories own storage and cryptography. No widget imports a database, a cipher or a file path.

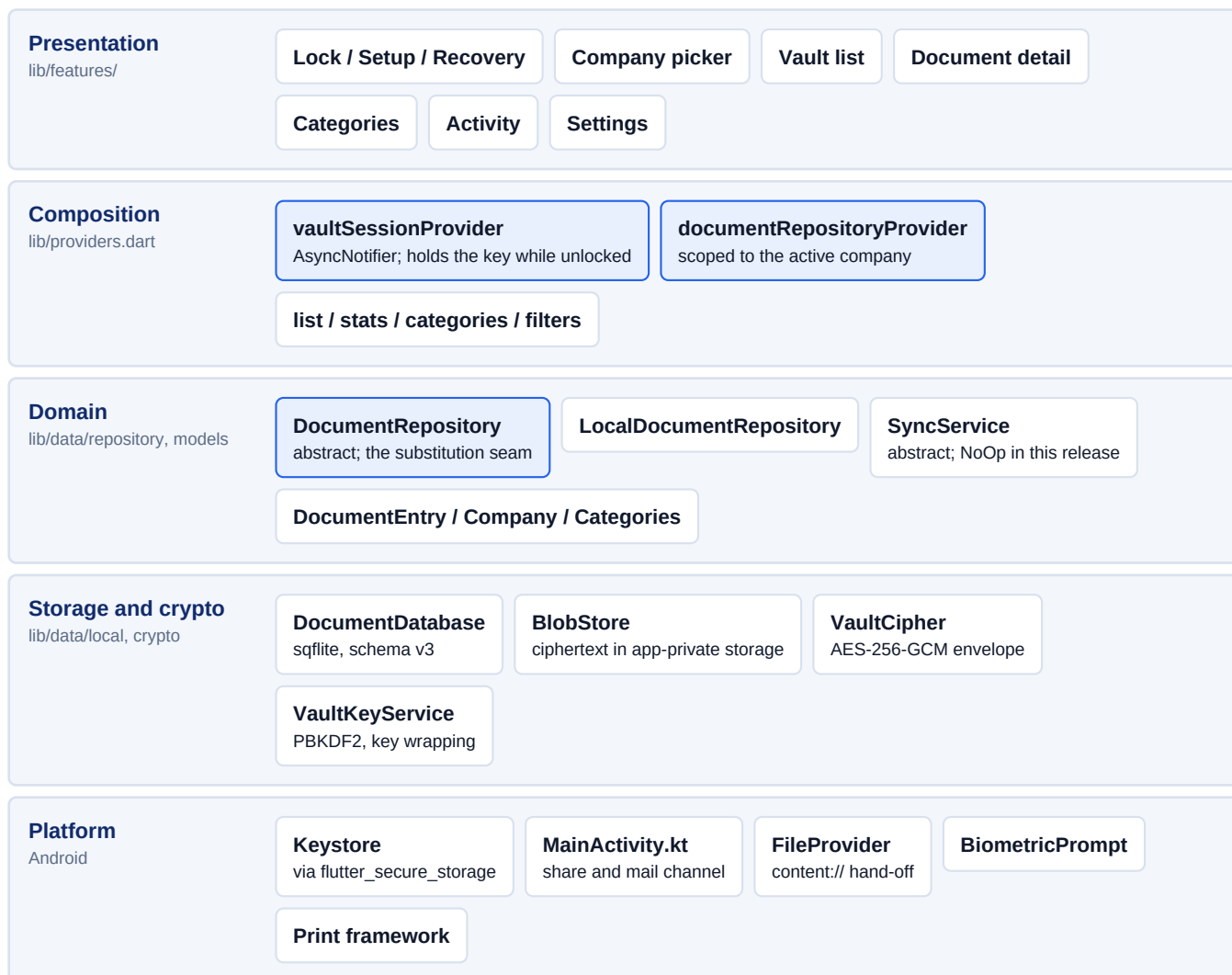


Figure 2. Layers. Each band depends only on the one below it. The repository interface is the seam at which an off-device backend would be substituted.

7.1 Source layout

```

lib/
  app/           MaterialApp, theme tokens, the gate
  core/          feature flags, formatting helpers
  data/
    crypto/      vault_cipher.dart, vault_key_service.dart
    local/       document_database.dart, blob_store.dart
    models/      document_entry.dart, company.dart, document_category.dart
    repository/  document_repository.dart (abstract), local_..._repository.dart
    services/    share_service.dart, print_service.dart
    sync/        sync_service.dart (abstract) + NoOpSyncService
  features/     one directory per screen
  providers.dart the only file where the layers meet
  main.dart

```

7.2 State management

Riverpod. A single AsyncNotifier holds the session: whether a vault exists, whether it is open, and the data key while it is. Everything else derives from it. The repository provider returns null while the vault is locked or no company is selected, so screens behind both gates never have to handle a null case that cannot occur.

A dependency rule that cost a defect

A provider must never be invalidated from inside a notifier that the provider watches. An early version of lock() invalidated the repository provider, which watches the session - a circular dependency. Riverpod asserts on this in debug builds only, so the release APK tolerated it silently. It was found by the first regression test written for the auto-lock fix. See section 13.

7.3 The gate

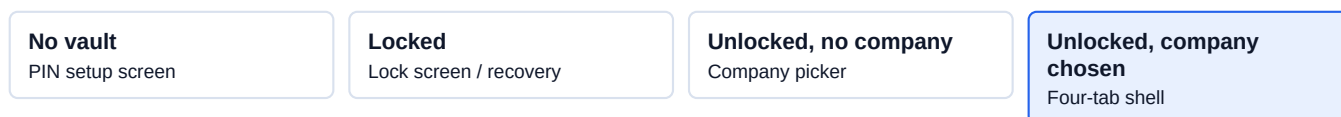


Figure 3. Top-level state. The gate watches the session and swaps the screen; no screen navigates to another of these.

Locking clears the active company as well as the key, so unlocking always returns to the picker rather than reopening whichever company happened to be selected when the screen went dark.

8 Technology stack

8.1 Toolchain

Component	Version	Note
Flutter	3.47.0 (stable)	Stable channel
Dart	3.13.0	Bundled with the SDK
JDK	Temurin 17.0.20+8	Portable install; no administrator rights were available on the build machine
Android Gradle Plugin	9.1.0	Kotlin DSL build scripts
Gradle	9.3.1	

Component	Version	Note
Kotlin	2.4.0	JVM target 17
Android SDK	platforms 34/35/36	build-tools 34.0.0 to 36.1.0
NDK	28.2.13676358	Required by the Flutter Gradle plugin and genuinely used by a transitive JNI dependency
CMake	3.22.1	Required by the same native build

8.2 Application dependencies

Fifteen direct dependencies, resolving to 126 packages in total. Each is present for a stated reason; none was added speculatively.

Package	Version	Why it is here
flutter_riverpod	2.6.1	State management. Async-aware providers hold the session and derive every screen from it.
cryptography	2.9.0	AES-256-GCM and PBKDF2-HMAC-SHA256. Pure Dart, so the same code is unit-testable off a device.
flutter_secure_storage	10.0.0	Android Keystore-backed storage for the wrapped keys. Pinned; see section 13.
sqflite	2.4.3	SQLite for the metadata index, with versioned migrations.
path_provider	2.1.6	Locates app-private directories for the blob store and the share staging area.
path	1.9.1	Path joining and basename sanitising.
file_picker	12.0.0	System file picker for import, and the save dialog for export.
local_auth	3.0.2	Fingerprint and face unlock via BiometricPrompt.
pdfx	2.4.7	PDF rendering. Bundles PDFium, which unlike Android's own renderer opens password-protected files.
printing	5.15.0	Hands a document to the Android print framework.
pdf	3.13.0	Wraps an image in a single-page PDF for printing. Also generates this report.
share_plus	13.3.0	System share sheet - the fallback when the direct WhatsApp intent does not resolve.
intl	0.20.3	Date and number formatting.
package_info_plus	10.2.1	Reads the real versionName baked into the APK for the Settings screen.
cupertino_icons	1.0.8	Icon set completeness.

8.3 Development dependencies

Package	Version	Purpose
flutter_test	SDK	Unit tests, and the binding the screenshot harness renders under.
flutter_lints	6.0.0	Lint set. The release gate refuses a build with any analyzer issue.
flutter_launcher_icons	0.14.4	Generates the launcher mipmaps and the adaptive-icon layers.

9 Core workflows

9.1 Unlocking

- 1 The user enters six digits**
Collected on a drawn keypad; the entry never reaches an IME.
- 2 PBKDF2-HMAC-SHA256 derives the key-encryption key**
Using the salt and iteration count stored with the vault, so an older vault keeps unwrapping after the default cost rises.
- 3 The wrapped data key is decrypted**
A SecretBoxAuthenticationError is translated to WrongPinException; the screen shows "Incorrect PIN".
- 4 The session holds the key in memory**
Never written back to disk. Dropped on lock, on backgrounding, and when the process ends.

Figure 4. Unlock. A wrong PIN fails as an authentication error on the GCM tag, not as a mismatch against a stored value.

9.2 Importing a document

- 1 The system file picker opens**
Wrapped in runTrusted, which suppresses auto-lock while another activity is in front. Without it the vault locks and the import is lost - see section 13.
- 2 The user confirms title, category, notes and lock**
All collected before any encryption starts, so a cancelled prompt writes nothing.
- 3 The bytes are sealed**
A locked document is encrypted first under the document-lock key, then under the vault key. On disk it is indistinguishable from an unlocked one.
- 4 The blob is written, then the metadata row**
Written to a .part name and renamed, so an interrupted write cannot leave a half-file the row already points at. If the insert fails, the blob is deleted and the error rethrown.

Figure 5. Import. Ordering is deliberate: an orphan blob can be pruned, a metadata row pointing at nothing cannot be repaired.

9.3 Reading, sharing and printing

- 1 Preview: decrypt to memory**
Bytes are handed to the renderer and never written to disk. A locked document prompts for its 4-digit PIN first; that key is held for the screen only, not cached in the session.
- 2 Share: stage one plaintext file**
Written into app-private cache under shared/, which is emptied before every share and at application start. Handed over as a content:// URI through a FileProvider - a file:// URI across an app boundary throws on Android 7+.
- 3 Print: hand a PDF to the framework**
PDFs pass through untouched to preserve text selection; images are wrapped in a single page. Office formats are refused before the dialog opens.
- 4 Export: the user chooses a destination**
Through the system save dialog. This is the one path that puts a decrypted copy into shared storage, and it is always explicit.

Figure 6. The three routes by which plaintext exists, and what bounds each.

Where the protection ends

Once a document has been shared, printed or exported, it is an ordinary file under the receiving application's rules. A document sent through a messaging app may be backed up by that app to its own cloud. DocVault controls what leaves and when, not what happens afterwards, and the documentation says so rather than implying end-to-end control.

10 Important engineering decisions

10.1 A random data key, wrapped by the PIN

- PROBLEM** Deriving the encryption key directly from the PIN would make every PIN change a full re-encryption of the vault, and would tie the strength of stored data to a six-digit secret in a way that cannot be changed later.
- DECISION** Generate a random 256-bit data key, and use the PIN only to wrap it.
- IMPLEMENTED** PBKDF2-HMAC-SHA256 at 210,000 iterations over a per-vault salt produces a key-encryption key; only the wrapped data key, the salt and the iteration count are persisted. The iteration count is stored rather than assumed, so raising the default later does not lock existing vaults out.
- RESULT** Changing the PIN re-wraps 60 bytes. A wrong PIN fails as a GCM authentication error rather than a hash comparison, so the stored blob reveals nothing about whether a guess was close.

10.2 Suppressing auto-lock across system activities

- PROBLEM** Dropping the key when the app leaves the foreground is the point of a vault. But the file picker, the save dialog, the biometric prompt, printing and sharing all start another activity, which pauses the app in exactly the same way. The first released build therefore locked itself mid-import: the user saw the PIN pad and no document was added.
- DECISION** Suppress auto-lock for interactions the user themselves initiated, and only those.
- IMPLEMENTED** A depth counter rather than a flag, so an export starting while an import sheet is open cannot re-arm the lock for the outer operation, with a timeout backstop in case a plugin never returns. Suppression is released on onResumed, not when the call returns - startActivity comes back as soon as the intent is dispatched, before the other app is in front.
- RESULT** Import, export, print and share work; an explicit "Lock now" is never suppressed; and eight regression tests pin the behaviour, including the two orderings that caused the original defects.

10.3 A recovery key instead of a password reset

- PROBLEM** Users expect a "forgot password" e-mail. With the key derived from the PIN and no server holding anything, there is nothing to reset against.
- DECISION** Provide an opt-in recovery key and describe the trade-off accurately, rather than implementing something that looks like a reset but weakens the vault.
- IMPLEMENTED** A 150-bit code in Crockford base32 groups - no I, L, O or U, so it can be read aloud or retyped - wraps a second copy of the data key. It is shown once and never stored. An optional recovery e-mail address lets the user send it to themselves from their own mail app; the application never sends mail itself, which would require either a server or SMTP credentials inside the APK.
- RESULT** A forgotten PIN is survivable if the user prepared for it. When they did not, the dialog says plainly that the vault cannot be opened and why, instead of offering a reset that would not work.

10.4 Choosing the PDF renderer by what it links

PROBLEM	A user reported that a PDF failed to preview with "Password required to access document" on a file they had not locked. Android's built-in PdfRenderer cannot open password-protected PDFs at all.
DECISION	Use a renderer that bundles PDFium rather than one that wraps the platform renderer - and verify that claim against the built artefact rather than the package description.
IMPLEMENTED	A first replacement package was rejected after inspecting the APK showed no libpdfium.so and its Android implementation was the platform renderer with the password argument ignored. The shipped choice, pdfx, was confirmed by listing libpdfium.so for all three ABIs in the built APK.
RESULT	Password-protected PDFs open after the user supplies the password, entirely on-device. The cost is APK size: the universal APK rose from 56.5 MB to 74.5 MB, which is why per-ABI APKs and the AAB exist.

10.5 Companies as an organisational, not cryptographic, boundary

PROBLEM	A requirement to keep separate document sets for separate businesses could be read as either a filter or a separate vault.
DECISION	Implement partitioning by company over one key, and document the limit explicitly rather than allowing it to be assumed.
IMPLEMENTED	A company_id column with its own index, seeded through a v2 to v3 migration, and a repository constructed per selected company so that no query can forget to scope itself. Anyone who unlocks the vault can switch between all companies.
RESULT	The organisational requirement is met at low cost. The report and the architecture document both state that per-company keys would be a different design, and that retrofitting them means re-encrypting every blob - much cheaper to decide before a vault fills up than after.

10.6 A release gate rather than a build command

PROBLEM	An APK built by hand at the end of a session is easy to ship with a failing test, an unformatted analyzer warning, or the debug signing key.
DECISION	Make the release script the only supported way to produce an artefact, and make it refuse to proceed on any failure.
IMPLEMENTED	scripts/build-release.ps1 runs pub get, analyze and the full test suite before building, warns loudly when key.properties is absent and the debug key would be used, copies artefacts into dist/<version>/ under names carrying the version, and writes a SHA256SUMS file.
RESULT	Ten releases produced the same way, each with its own directory and checksums. A stale or unsigned build is difficult to ship by accident.

11 AI-assisted development

This project was built through an AI-assisted workflow, and the report states that plainly because it affects how the work should be read. Implementation code was produced by Claude Code (Anthropic) in interactive terminal sessions. Requirements, product decisions, acceptance testing on real hardware and final responsibility for the result were the author's throughout.

11.1 Division of work

Owned by the author	Produced with AI assistance
The requirement, the scope and every change of scope	Dart, Kotlin and Gradle implementation
Acceptance testing of each build on a physical Android device	Diagnosis of build failures from error output
Defect reports from real use, in the user's own terms	Root-cause analysis and the fixes
Product decisions - wording, ordering, which features to add	Unit tests, including regressions for each reported defect
Judging whether a delivered change was actually fixed	Documentation, build scripts and this report
The decision to accept or reject proposed approaches	Toolchain installation and environment repair

11.2 How the loop actually ran

- 1 The author states a requirement or reports a defect**
In plain terms and from real use - for example, that adding a document asked for the PIN and then added nothing.
- 2 The assistant reproduces, diagnoses and implements**
Including reading the platform behaviour behind a symptom rather than patching the symptom.
- 3 The change is gated automatically**
Analyzer clean and the full test suite green before an artefact is produced. A regression test is written for each defect, naming the symptom.
- 4 The author installs the signed build and judges it**
On their own device. This is the only step that exercises the Keystore, the file picker, printing and WhatsApp - none of which can be tested off a device.

Figure 7. The iteration that produced ten releases.

11.3 Where AI assistance needed checking

Two cases in this project are worth recording, because both would have shipped as working code that did not work.

- A PDF package was selected on the strength of its description, which implied it could open password-protected files. Inspecting the built APK showed no PDFium binary, and its Android implementation was the platform renderer with the password argument ignored. It was replaced before release, and the replacement was confirmed by listing the shipped native libraries.
- A circular provider dependency compiled cleanly and ran without complaint in the release build, because the framework asserts on it only in debug. It surfaced only when a regression test was written for an unrelated fix.

The working rule that came out of this

A clean compile and a green test run prove that code builds and that the logic under test behaves. They do not prove that the artefact contains what it should, that a dependency does what its description says, or that the application works on a phone. Those were verified separately - by inspecting the APK's manifest, signature and native libraries, and by installing each build on a real device. Section 14 lists exactly which claims rest on which kind of evidence.

12 Implementation details

12.1 The encryption envelope

Every encrypted blob is self-describing. Keeping the nonce and tag with the bytes, rather than in the database, means a blob can be verified - or later uploaded - without a matching metadata row, which is what makes off-device backup a substitution rather than a schema change.

```
[ 12-byte nonce ][ ciphertext ][ 16-byte GCM tag ]
                        28 bytes of overhead per document
```

A locked document is the same envelope applied twice:

```
plaintext --AES-GCM(DLK)--> inner --AES-GCM(DEK)--> blob on disk
```

Decryption of a wrong key or an altered blob raises `SecretBoxAuthenticationError`. Callers treat both as "this document is unreadable" rather than trying to distinguish them.

12.2 Data model

SQLite holds metadata only; document bytes never appear in the database. The schema is at version 3, with migrations written from the first release rather than retrofitted.

```
documents
  id          TEXT PRIMARY KEY
  company_id  TEXT NOT NULL DEFAULT 'default' -- v3; indexed
  title       TEXT NOT NULL
  category    TEXT NOT NULL                  -- indexed
  original_file_name TEXT NOT NULL          -- shown, and used to pick an
                                           -- extension on export; never a path
  mime_type   TEXT NOT NULL
  size_bytes  INTEGER NOT NULL              -- plaintext length
  added_at    INTEGER NOT NULL              -- indexed DESC
  updated_at  INTEGER
  blob_name   TEXT NOT NULL UNIQUE
  notes       TEXT
  is_locked   INTEGER NOT NULL DEFAULT 0    -- v2
  sync_state  TEXT NOT NULL DEFAULT 'localOnly' -- reserved; indexed
  remote_id   TEXT                         -- reserved
  synced_at   INTEGER                       -- reserved

categories
  name      TEXT PRIMARY KEY
  created_at INTEGER NOT NULL

companies
  id      TEXT PRIMARY KEY
  name    TEXT NOT NULL UNIQUE
          COLLATE NOCASE
  created_at INTEGER NOT NULL

-- user-added only; the
-- built-ins are constants
```

Migration	Change	Note
v1	Initial schema	The three sync columns were present from the start, so enabling off-device backup later is not a migration of every row.
v1 -> v2	Document lock and user categories	Adds <code>is_locked</code> and the categories table. Also rewrites category values from the old enum name ("identity") to the label ("Identity"), so custom categories need no separate identifier.
v2 -> v3	Company partitioning	Adds <code>company_id</code> with a literal default - SQLite rejects a non-constant DEFAULT in ALTER TABLE ADD COLUMN - plus its index, the companies table, and a seeded default company so the picker is never empty.

12.3 Blob storage

- Ciphertext lives under the application-support directory, not in shared storage, so nothing is visible to other applications or to a file manager.
- Writes go to a .part name and are renamed into place, so an interrupted import cannot leave a half-written blob that a metadata row already points at.
- pruneOrphans removes blobs with no metadata row - the residue of a failed import or a delete that died between the two stores.

12.4 Android platform integration

Two capabilities are not covered by Flutter plugins and are implemented in 152 lines of Kotlin behind a single method channel: handing a file to WhatsApp directly rather than through the system chooser, and opening the mail application with a message already composed.

```
MethodChannel("com.docvault.app/share")
  isWhatsAppInstalled -> queries com.whatsapp, then com.whatsapp.w4b
  shareDocument      -> FileProvider content:// URI
                      + explicit grantUriPermission to the target
                      + setPackage(...), falling back to a chooser
                      on ActivityNotFoundException
  composeEmail       -> ACTION_SENDTO with a mailto: scheme, so only
                      mail apps match

AndroidManifest.xml
<queries> com.whatsapp, com.whatsapp.w4b, SENDTO/mailto, SEND */*
Android 11+ hides installed packages unless declared. Without this,
setPackage("com.whatsapp") resolves to nothing and the share silently
fails on a device that has WhatsApp installed.
```

The direct intent is treated as an optimisation, not a dependency: any failure falls through to share_plus and the system sheet, so the user always has a way to send the document. A share button that greys itself out, or does nothing at all on a device without WhatsApp, is indistinguishable from a broken one.

13 Challenges and solutions

13.1 Build environment

The build machine had no Android toolchain, 7.9 GB of RAM, no administrator rights, and the project on a different drive from the package cache. Each failure below produced a misleading error, and each is now documented in docs/DEPLOYMENT.md so that it is diagnosed in minutes rather than hours on the next machine.

Symptom	Actual cause	Resolution
Package ndk not found; sdkmanager exits with 0xC0000409	Command-line tools 23.0 replaced sdkmanager with an "android sdk" CLI whose package paths use / not ;. The old .bat wrapper lets the shell split on ; and passes two invalid package names.	Install packages explicitly with the new CLI, and set android.builder.sdkDownload=false so a build fails loudly on a missing component instead of silently fetching one.
Could not close incremental cachestab	Two separate causes. The Flutter template requests 12 GB of JVM memory on an 8 GB machine; and Kotlin incremental compilation relativises plugin sources against the project directory, which is impossible when the project is on E: and the package cache on C:.	Size the JVM for the machine, and set kotlin.incremental=false. Both are documented in android/gradle.properties with the alternative fix.

Symptom	Actual cause	Resolution
flutter_secure_storage requires Android SDK version 37	Version 11 of the plugin targets an Android platform that is not published yet, so no amount of upgrading satisfies it.	Pin to 10.x, with a comment recording why and what to check before unpinning.
Keystore file not found for signing config	Inside the :app module a bare file() resolves against android/app/, so a storeFile of "../keystore/..." looked in the wrong place.	Resolve with rootProject.file(), which is what key.properties documents.
key.properties silently unreadable; sha256sum -c fails	PowerShell 5.1 writes a UTF-8 byte-order mark, which java.util.Properties and sha256sum both choke on.	Write files with [System.IO.File]::WriteAllText instead of Set-Content.
Script indexes the first character of a filename	Where-Object returning a single item unwraps to a bare string, so [0] indexes a character rather than an element.	Wrap in @(...). The same trap appeared twice.

13.2 Application defects

Four were reported by the author from real use of an installed build; two were found by the test suite. Each has a regression test that names the symptom.

Defect	Root cause	Fix
Adding a document asked for the PIN and then added nothing	The system file picker pauses the activity. The lifecycle handler treated that as a backgrounding, dropped the key and disposed the screen that was mid-import.	runTrusted: a depth-counted suppression of auto-lock across user-initiated system activities, with a timeout backstop.
Sharing to WhatsApp did nothing	Two causes at once. The lock handler deleted the staged plaintext while WhatsApp was still reading it; and suppression ended when startActivity returned, which is before Android delivers onPause, so the vault locked during the hand-off.	Hold suppression until onResume actually fires, and move staged-file cleanup to application start and to just before each share.
PDF preview reported "Password required" on an unlocked document	The file was genuinely password-protected. Android's PdfRenderer cannot open such files, and the wrapper package first chosen accepted a password argument and ignored it.	Switch to a renderer that bundles PDFium, verified by listing libpdfium.so in the built APK for all three ABIs.
Circular provider dependency (found by test)	lock() invalidated a provider that watches the session. Riverpod asserts on this in debug only, so the release build tolerated it silently.	Remove the invalidation; watching already rebuilds the dependent provider when the session changes.
Custom categories appeared after "Other"	"Other" was treated as an ordinary built-in rather than as the catch-all and the fallback for a removed category.	Order explicitly: built-ins, then user categories, then "Other" last. Covered by a test.
Auto-lock never re-armed after a cancelled picker (found by test)	An early version released suppression only on success, so an exception left auto-lock disabled for the session.	Release in a finally block; a regression test asserts the lock re-arms after the action throws.

14 Testing and verification

14.1 Automated tests

60 tests across 5 suites, 812 lines. They target the logic where the defects actually were - the session state machine - rather than spreading thinly across widgets.

Suite	Tests	What it covers
vault_session_test.dart	21	Unlock and wrong PIN; the eight auto-lock suppression cases including both orderings that caused shipped defects; document-lock setup and key reuse; recovery key creation, normalisation and failure; biometric enable and unlock.
document_entry_test.dart	15	Row mapping in both directions, including defaults for missing columns, blank-category fallback, locked flag, and sync state parsing.
vault_cipher_test.dart	12	Round trip; envelope shape and overhead; rejection of a wrong key, a truncated blob and a tampered byte; nonce uniqueness across encryptions.
formatting_test.dart	9	Byte and relative-date formatting at boundary values.
company_test.dart	3	Name normalisation, default identification, equality.

14.2 The release gate

```

.\scripts\build-release.ps1

1 flutter pub get
2 flutter analyze      -> any issue aborts the build
3 flutter test         -> any failure aborts the build
4 flutter build ...    -> apk | split | aab | all
5 copy to dist\<version>\ with version-carrying names
6 write SHA256SUMS.txt -> BOM-free, re-read so a partial rebuild
                        does not drop entries for other artefacts

```

14.3 Artefact verification

A clean compile does not prove that the shipped artefact contains what it should. These checks were run against built APKs, and one of them caught a feature that would have failed on every device.

Check	Command	What it establishes
Signature	apksigner verify --print-certs	The APK is signed with the intended certificate and not with the debug key.
Manifest	apkanalyzer manifest print	The merged manifest carries the expected label, backup flags, FileProvider and package queries - and no INTERNET permission.
Native libraries	unzip -l <apk> grep .so	libpdfium.so is present for all three ABIs. This is the check that rejected an earlier PDF package whose description implied a capability it did not ship.
Size	ls dist/<version>/	Per-ABI and bundle sizes, tracked across releases.

14.4 Interface rendering

tool/screenshots/capture_test.dart renders the eight screens in section 6 through the real widget tree under the flutter_test binding. It is kept in tool/ rather than test/ so the release gate does not rewrite the images on every build. It is a documentation tool, not a golden test: it asserts nothing, and would not catch a visual regression on its own.

14.5 What is not covered

- No widget or integration tests for the screens themselves. Rendering is exercised by the screenshot harness, but nothing asserts on the result.
- No instrumented tests on a device or emulator. The Kotlin method channel, the FileProvider hand-off, BiometricPrompt, the print framework and the file picker are therefore unverified by automation.
- The Android Keystore cannot be reached off a device, so VaultKeyService is tested through a fake in the session tests; its real storage path is exercised only by running the application.
- All on-device verification was manual, performed by the author on their own handset across ten builds. It is not reproducible from the repository.

15 Security, reliability and data integrity

15.1 What the design protects against

Scenario	Outcome
Another application reads the storage directory	It finds AES-256-GCM ciphertext in an app-private directory and no key.
The device is lost while locked	The data key exists only in memory and only while unlocked. A powered-off or locked device holds nothing usable.
The secure-storage blob is extracted	It contains the wrapped key, a salt and an iteration count. Unwrapping requires the PIN, and the cost of guessing is 210,000 PBKDF2 iterations per attempt.
A stored blob is altered on disk	GCM authentication fails and the document is reported unreadable rather than silently returning corrupt bytes.
The vault is restored to another device from a backup	allowBackup and fullBackupContent are disabled and data extraction rules are declared, so the vault is excluded from Android automatic backup.
An attacker inspects network traffic	There is none. The application declares no INTERNET permission and contacts nothing.

15.2 What it does not protect against

- A compromised or rooted device. If the operating system is not trustworthy, nothing above it can be.
- Someone who knows the PIN, or who is handed an unlocked phone. The per-document lock is a second gate against the latter, not an independent cryptographic boundary: four digits is roughly 13 bits, and its value comes from sitting behind the vault PIN and the Keystore.
- Anything that happens to a document after it is shared, printed or exported. At that point it is an ordinary file under another application's rules.
- Loss of both the PIN and the recovery key. This is unrecoverable by design, and the interface says so.

15.3 Reliability and data integrity

Risk	Handling
Interrupted write leaves a half-file	Blobs are written to a .part name and renamed into place.
Metadata row points at a missing blob	The blob is written before the row; if the insert fails the blob is deleted and the error rethrown.
Blob with no metadata row	pruneOrphans removes it; the vault reset path uses the same mechanism.

Risk	Handling
Schema change on an existing install	Versioned migrations from v1, each tested against the mapping in <code>document_entry_test.dart</code> .
Duplicate company or category names	Normalised whitespace and a NOCASE unique constraint, so "Acme Ltd" and "acme ltd" are one entity rather than two that look identical.
Crafted filename escaping the staging directory	Share staging strips path separators with basename before writing.
Last company deleted, leaving documents homeless	Deleting the only remaining company is refused in the interface.

Secrets and this repository

The signing keystore and `android/key.properties` are excluded from version control, and no credential appears in the source, in this report, or in any build artefact. The application holds no API key, no SMTP credential and no server address, because it contacts no service.

16 Deployment and usage

16.1 Artefacts

Artefact	Size	Use
DocVault-1.6.2-10-universal.apk	74.5 MB	The link handed to a client. Installs on any device; carries native libraries for all three ABIs.
...-arm64-v8a.apk	27.1 MB	Every modern Android phone. The smallest sensible direct download.
...-armeabi-v7a.apk	23.0 MB	Older 32-bit devices.
...-x86_64.apk	28.8 MB	Emulators and the few x86 devices.
DocVault-1.6.2-10.aab	65.5 MB	Google Play only. Play splits it by language, density and ABI, so the delivered download is smaller than any APK here.

16.2 Signing

Release signing is read from `android/key.properties`, which is not in version control. When it is absent - a fresh clone, or a CI job without secrets - the build falls back to the debug key so that the build still succeeds, and logs a warning saying the result is installable but must not be published. The signing path was exercised end to end rather than left untested.

An open decision before any public release

The keystore in the repository working tree was generated during setup so the signing path could be verified, not chosen by the author. Before a first Google Play upload it must either be adopted and backed up durably, or deleted and regenerated with `scripts/create-keystore.ps1`, which prompts for a password the author chooses. After a Play release the key is permanent: losing it means never being able to update that listing. The application ID `com.docvault.app` is permanent in the same way and should be changed if the domain is not the author's.

16.3 Getting it to a non-technical user

- 1 Build and publish**
build-release.ps1 produces the APK and its SHA-256. The APK and the download page are uploaded to any static host; the server must send Content-Type application/vnd.android.package-archive or some browsers save it as a .zip that will not install.
- 2 The user opens the link on their phone**
The page states the version, size and checksum.
- 3 The user allows installation from this source once**
Android prompts for it; docs/CLIENT_INSTALL.md walks through the exact wording, including the "App not installed" cases and what causes them.
- 4 First run: set a PIN, then a recovery key**
The install guide puts the recovery key first, because it is the only step that cannot be done later if the PIN is forgotten.

Figure 8. Delivery. distribution/download.html is a self-contained page - no CDN, no fonts, no tracking - that hosts next to the APK on any static host.

16.4 Command reference

```
. .\scripts\setup-env.ps1      put the toolchain on PATH
flutter analyze               must be clean
flutter test                  60 tests
flutter build apk --debug     installs alongside the release build

.\scripts\bump-version.ps1 -Part minor
.\scripts\build-release.ps1 universal APK, gated
.\scripts\build-release.ps1 -Target aab
.\scripts\build-release.ps1 -Target all

.\scripts\create-keystore.ps1 new signing key; prompts for password
.\tool\generate_icon.ps1      regenerate launcher art
flutter test tool/screenshots re-capture the section 6 screens
dart run tool/generate_report.dart regenerate this report

apksigner verify --print-certs <apk> confirm the signing certificate
unzip -l <apk> | grep '\.so$'    confirm native libraries shipped
```

16.5 Documentation shipped with the project

Document	Lines	Audience
docs/DEPLOYMENT.md	408	Whoever builds and ships. Machine setup, building, versioning, signing, hosting, Play Store, and a troubleshooting section keyed by the exact error text.
docs/ARCHITECTURE.md	440	Whoever extends it. Layering, the security model in full, and the seam for each planned feature.
docs/CLIENT_INSTALL.md	152	The end user. Plain-language install, recovery-key setup, and what to do when something fails.
README.md	105	Orientation and quick start.

17 Role and contribution

I specified, directed, tested and own this project. The implementation was written with AI assistance under that direction, and section 11 describes that process in detail. This section states what is mine so the rest of the report can be read accurately.

- I defined the requirement: an offline encrypted document vault a non-technical person could install from a link, and the scope that followed from it.
- I made the product decisions. Multiple companies, the per-document lock, custom categories, direct WhatsApp sharing, in-app preview for every viewable format, and the interface direction were all my calls, as were smaller ones such as category ordering and button wording.
- I tested every build on my own Android device. All four user-reported defects in section 13 came from that testing, described in terms of what the application did rather than what the code did.
- I judged completion. A change was finished when it worked on my phone, not when it compiled or when the tests passed.
- I accepted or rejected proposed approaches, including accepting a recovery key in place of the password-reset e-mail I originally asked for, once the reason it could not be built was explained.
- Where a decision is still open - the signing keystore, the application ID, per-company keys - it is open because it is mine to make, and section 16 and section 20 record it as such.

Stated plainly

I did not hand-write the Dart, Kotlin and Gradle source in this repository. It was generated by an AI assistant working to my requirements, and I verified the result on real hardware. The engineering judgement in this report - what to build, what to refuse, what counts as done, and what remains unproven - is recorded honestly so the work can be assessed for what it is.

18 Results and verified metrics

Every figure below was measured from the source tree or the build output at version 1.6.2, using the commands recorded in `tool/generate_report.dart`.

Metric	Value	How it was measured
Application source	38 files, 7551 lines of Dart	<code>find lib -name "*.dart"</code>
Platform source	152 lines of Kotlin	<code>MainActivity.kt</code>
Tests	60 tests in 5 suites, 812 lines	<code>flutter test</code>
Static analysis	No issues	<code>flutter analyze</code> , over lib, test and tool
Dependencies	15 direct, 126 resolved	<code>pubspec.yaml</code> and <code>pubspec.lock</code>
Files under version control	97	<code>git ls-files</code>
Database schema	Version 3, two migrations	<code>document_database.dart</code>
Releases produced	10, from 1.0.0+1 to 1.6.2+10	<code>dist/</code>
Universal APK	74.5 MB	<code>dist/1.6.2+10/</code>
Smallest per-device APK	23.0 MB (armeabi-v7a)	<code>dist/1.6.2+10/</code>
Play bundle	65.5 MB	<code>dist/1.6.2+10/</code>
Documentation	1,105 lines across four documents	<code>wc -l docs/*.md README.md</code>

18.1 Artefact size across releases

Version	Universal APK	What changed
1.0.0+1	51.2 MB	First release: vault, PIN, import, preview, categories, export.
1.0.1+2	51.2 MB	Auto-lock suppression - the import defect.
1.1.0+3	51.5 MB	Per-document lock, custom categories.
1.2.0+4	51.6 MB	Metadata behind an info action; interface refinements.
1.3.0+5	55.5 MB	Print and direct WhatsApp sharing; interface rebuilt to the agreed design tokens.
1.4.0+6	55.6 MB	Recovery key, recovery e-mail address, working "Forgot PIN".
1.5.0+7	55.9 MB	Multiple companies; schema v3.
1.6.0+8	56.5 MB	Preview for every viewable format.
1.6.1+9	74.5 MB	PDFium bundled so password-protected PDFs open. The size rise is this change.
1.6.2+10	74.5 MB	WhatsApp hand-off race fixed.

18.2 Outcome against the objectives

Objective	Outcome
Encrypted at rest with a key the owner controls	Met. AES-256-GCM over whole documents; key wrapped under a PBKDF2-stretched PIN and held by the Android Keystore.
Entirely offline	Met. No INTERNET permission in the shipped manifest; verified in the merged manifest of a built APK.
Protected documents remain usable	Met for images, PDF and text - viewable, printable, shareable and exportable. Office formats can be stored and shared but not previewed; see section 19.
Installable by a non-technical user from a link	Met. Signed APK, self-contained download page, and a plain-language install guide. Installation was performed this way on the author's device for each release.
Repeatable, verifiable releases	Met. Ten gated releases, each with versioned artefacts and checksums.
Open to later cloud features without a rewrite	Partially met by design rather than by demonstration. The interfaces, feature flags and schema columns exist; no backend has been built against them, so the claim is structural, not proven.
Accurate with the user about the limits	Met. The absence of a PIN reset, the meaning of the document lock, and the consequences of sharing are stated in the interface and in the install guide.

19 Limitations

These are current and known. Several are consequences of the design rather than defects, but all of them affect what a user can expect.

Limitation	Detail
No backup of any kind	The vault exists on one device. If the phone is lost, reset, or the application uninstalled, the documents are gone. Exporting copies is the only mitigation, and it is manual.

Limitation	Detail
A forgotten PIN with no recovery key is terminal	Nothing stores the PIN and no server holds anything. The only remaining option is erasing the vault.
The document lock is a second gate, not a second boundary	Four digits is roughly 13 bits. Its value comes from sitting behind the vault PIN and the Keystore, not from its own strength.
Companies share one encryption key	Partitioning is organisational. Anyone who unlocks the vault can open every company in it.
Biometric unlock is not a second factor	It releases a second copy of the same key after BiometricPrompt succeeds. It is a convenience over the same secret.
Word and Excel files cannot be previewed	They can be stored, shared, exported and locked, but rendering them offline would mean shipping a converter or uploading the file. The application shows an explanation instead of a blank page.
Sharing, printing and exporting produce plaintext	Necessarily so. What the receiving application does with it - including backing it up to its own cloud - is outside DocVault's control.
The universal APK is large	74.5 MB, because PDFium ships for three ABIs. Per-ABI APKs (27.1 MB for arm64) and the Play bundle exist precisely for this reason.
No automated interface or on-device testing	Screens are rendered but nothing asserts on the result; the platform channel, biometrics, printing and the file picker are verified only by manual use.
Not published, and not ready to publish today	A Play listing additionally needs a privacy policy URL, a decision on the application ID, and a decision on the signing key. See section 16.
Screenshots are renders, not device captures	They show the shipping interface code accurately but cannot evidence runtime behaviour below the widget layer.

20 Future improvements

Each of these is grounded in a seam that already exists in the code, and each is listed with the work it actually requires rather than as an aspiration.

Improvement	What already exists	What it needs
Encrypted cloud backup	An abstract SyncService with a no-op implementation, and a contract that blobs upload exactly as they sit on disk - already sealed.	A backend or a provider integration. The key never leaves the device, so a restore on a new device needs a passphrase-wrapped copy of the key, which is a deliberate and documented weakening.
Google Drive or Dropbox as the backend	The same interface.	OAuth per provider. The user's own cloud holds only ciphertext, which is both the feature and the support burden.
Multi-device sync	sync_state, remote_id and synced_at columns, present since v1.	Conflict handling, and tombstones - without them a document deleted on one device returns from another.
User accounts	An enableAccounts flag and a gate for an AuthService.	Identity kept separate from the encryption key, or losing an account locks the user out of documents already on their device.
Per-company encryption keys	Nothing; it would extend VaultKeyService.	Re-encrypting every existing blob. Substantially cheaper to decide before vaults fill up than after.

Improvement	What already exists	What it needs
Automated interface tests	A harness that already renders every screen through the real widget tree.	Golden comparison and assertions, plus instrumented tests on a device for the platform channel and biometrics.
Play Store release	A configured AAB with language, density and ABI splits.	A privacy policy URL, a store listing, and the two permanent decisions in section 16.
Per-company custom categories	A categories table that is currently vault-wide.	A company_id column and a migration - small now, and easier before users create many categories.

21 Conclusion

DocVault meets the requirement it was built for: documents on an Android phone are stored as authenticated ciphertext, reachable only through the application after a PIN or biometric unlock, with no account and no network involvement at any point. It remains usable while protected - previewable, printable, shareable and exportable - which was the part most likely to be sacrificed and the part that decides whether a user keeps an unprotected copy alongside it.

Ten gated releases were produced, four defects reported from real use were diagnosed to root cause and fixed with regression tests, and two further problems were caught by the test suite and by inspecting the built artefact rather than by trusting a clean compile. The architecture keeps its layers apart deliberately, so cloud backup, accounts and sync can be added at an interface rather than through a rewrite - a claim that is structural and stated as such, not demonstrated.

What the project does not do is equally clear, and deliberately so. There is no backup, a forgotten PIN without a recovery key is terminal, the per-document lock is a convenience gate rather than an independent cryptographic boundary, and nothing below the widget layer is covered by automated tests on a device. Those limits are stated in the application, in the install guide and in section 19 of this report, for the same reason throughout: a security tool that overstates what it protects is worse than one that is honest about where the protection ends.

22 Master prompt

A brief that reconstructs this project from an empty directory. The substance is one specification, given once in 22.1. Sections 22.2 to 22.7 wrap that specification in the prompt structure each model expects: paste the whole of 22.1 wherever the wrapper marks [SPECIFICATION], and the result is a complete, self-sufficient prompt for that model.

What it will and will not reproduce

This specifies behaviour, structure and constraints, not source text. An agent following it should arrive at the same architecture, the same security model and the same feature set, but not at line-for-line identical code. The parts that matter most are the hard constraints in A and the traps in H: those were paid for in failed builds and user-visible defects, and an implementation that skips them will rediscover every one.

22.1 The specification

=== A. GOAL AND HARD CONSTRAINTS ===

Build "DocVault": an offline-first encrypted document vault for Android, delivered as a signed APK that a non-technical person installs from a download link. Flutter and Dart. Support Android 7.0 (API 24) and above.

Hard constraints, none of them negotiable:

- * Nothing leaves the device. No analytics, no crash reporting, no INTERNET permission in the shipped manifest, no cloud dependency at runtime.
- * Documents are encrypted at rest with AES-256-GCM. Plaintext reaches disk only when the user explicitly exports, prints or shares.
- * The encryption key is wrapped by a PIN-derived key and held in Android Keystore-backed storage. It is never stored in the clear and never leaves the device.
- * Usable by someone who is not technical. Prefer plain language over precision where the two conflict, but never claim a protection the code does not provide.
- * Be accurate in the interface about what cannot be done rather than pretending. A forgotten PIN with no recovery key is unrecoverable: say so in those words.

Deliver alongside the application: a signed release APK, an AAB for a future Play listing, a deployment guide, a plain-language install guide, an architecture document, and build scripts that gate a release on tests.

=== B. ARCHITECTURE ===

Layer the code, and enforce the layering:

lib/app/	MaterialApp, theme, the gate that picks the screen
lib/core/	feature flags, formatting helpers
lib/data/crypto/	key service and cipher
lib/data/local/	sqflite database, encrypted blob store
lib/data/models/	entities
lib/data/repository/	an abstract DocumentRepository plus a local impl
lib/data/services/	share, print
lib/data/sync/	an abstract SyncService with a no-op implementation
lib/features/	one directory per screen
lib/providers.dart	the ONLY file where the layers meet

Rules:

- * Widgets depend on the repository interface, never on the local implementation, so a sync-aware repository can replace it untouched.
- * Declare SyncService now and ship NoOpSyncService. Its contract: blobs upload exactly as they sit on disk, already sealed, so no backend ever receives plaintext or the key.
- * Give the document entity sync_state, remote_id and synced_at columns in the first schema, so enabling sync later is not a migration of every row.

State: Riverpod. An AsyncNotifier holds the session - whether a vault exists, whether it is open, and the key while it is. Providers for the repository, document list, filters, sort, statistics and categories derive from it. The repository provider returns null while locked or while no company is selected. NEVER invalidate a provider from inside a notifier that the provider watches: that is a circular dependency, Riverpod asserts on it in debug builds only, and a release build will hide it.

Gate: no vault -> PIN setup. Locked -> lock screen. Unlocked with no company chosen -> company picker. Otherwise -> the four-tab shell. Locking clears the selected company as well as the key.

=== C. SECURITY MODEL ===

Key hierarchy:

- PIN --PBKDF2-HMAC-SHA256, 210,000 iterations, per-vault salt--> KEK
- KEK --AES-GCM unwrap--> DEK (random 256-bit data key)
- DEK encrypts every document blob

Store only the wrapped DEK, the salt and the iteration count. Never the PIN, never the unwrapped DEK. Persist the iteration count so raising the default later does not lock existing vaults out. Hold the DEK in memory only while unlocked. A wrong PIN must fail as a GCM authentication error, not as a comparison against a stored hash.

Envelope on disk: [12-byte nonce][ciphertext][16-byte GCM tag].

Optional biometric unlock: a second copy of the DEK released once local_auth succeeds, removed the moment biometrics are disabled. It is a convenience over the same key, not a second factor - document it as such.

Document lock: a separate 4-digit PIN deriving its own key, applied as an inner AES-GCM layer inside the vault layer. Do not cache that key in the session; hold it only for the screen that needs it, so a locked document stays locked while the vault is open. State plainly that this is a second gate against someone handed an unlocked phone, not an independent boundary.

Recovery: a 150-bit key in Crockford base32 groups (no I, L, O or U), shown once, wrapping a second copy of the DEK and able to set a new PIN. Present it as the only way back. Do NOT build a "reset link by e-mail": with the key wrapped by the PIN there is nothing to reset against, and any mechanism that could reset it without the PIN would defeat the encryption. An optional recovery e-mail address may be stored so the user can send the key to themselves from their own mail app; the application must never send mail itself, which would need a server or SMTP credentials inside the APK.

Auto-lock: drop the key when the application leaves the foreground. Read trap 4 in section H before implementing it.

=== D. DATA MODEL AND FEATURES ===

sqlite, with versioned migrations from the first release.

```
documents(id, company_id, title, category, original_file_name, mime_type,
          size_bytes, added_at, updated_at, blob_name, notes, is_locked,
          sync_state, remote_id, synced_at)
categories(name, created_at)      user-added only; built-ins live in code
companies(id, name, created_at)  name UNIQUE COLLATE NOCASE
```

Index added_at DESC, category, sync_state and company_id. Ciphertext lives in app-private storage under blob_name; the database holds metadata only. Write blobs to a .part name and rename into place. Write the blob before the metadata row, and delete it if the insert fails.

Screens:

- * PIN setup, lock screen, forgot-PIN / recovery.
- * Company picker: cards with document counts; add, rename, delete. Never allow deleting the last one.
- * Vault: status card, search, category chips, sort, document cards, FAB.
- * Add-to-vault sheet: title, category, notes, lock toggle. Collect all of it before encrypting, so a cancelled prompt writes nothing.
- * Document detail: the document fills the body; print, share, save a copy and an overflow menu (info, rename, change category, lock, delete) in the app bar.
- * Categories, Activity and Settings tabs behind a bottom navigation bar, in an IndexedStack so each tab keeps its scroll position.

Derive the activity feed from the timestamps already on each document rather than from a separate log, so it cannot disagree with the vault.

Preview every type that can be rendered offline: images directly, text and CSV decoded, PDF through a library that BUNDLES PDFium. Word and Excel have no offline renderer - show a plain explanation, and do not upload the file to a converter to work around it.

Share: hand the file to WhatsApp directly when installed, falling back to the system share sheet when not. That needs a Kotlin method channel, a FileProvider content:// URI, an explicit grantUriPermission to the target, and a <queries> entry for the WhatsApp packages - without <queries> the package is invisible on Android 11+ and the send silently resolves to nothing. Stage exactly one decrypted file at a time in app-private cache.

Print: PDFs pass through untouched to preserve text selection; images are wrapped in a single page; formats that cannot be rendered are refused before the dialog opens.

=== E. INTERFACE ===

Dark-first, professional, calm. Tokens, defined once:

```
primary #6F9EFF    accent #5CB8FF    success #4FD1A5
warning #F4C76A    danger #FF6B78
background #080D18 card #121C2E  border #263550
text #F5F7FC / #9AA8BF
```

Provide light-mode counterparts so the application stays usable for anyone running the system light. Type scale: display 32/700, title 22/600, headline 17/500, body 15/400, caption 12/400. Cards 18px radius, 1px border, no elevation. Buttons 14px radius, 52px tall.

Categories take fixed colours so the list stays scannable: Identity blue, Financial green, Medical red, Education cyan, Legal amber, Work violet. Custom categories hash into the same palette so a new one never introduces a colour the rest of the application does not use. "Other" is always last.

Draw the lock-screen mark rather than scaling the launcher icon, which is sized for a launcher grid and goes soft when enlarged. Use a drawn numeric keypad for PIN entry, not the system keyboard.

=== F. ANDROID BUILD AND RELEASE ===

applicationId com.docvault.app, minSdk 24, portrait only,
android:label "DocVault", allowBackup false, fullBackupContent false,
data extraction rules declared, and no INTERNET permission.

Read release signing from key.properties and resolve the store file with rootProject.file(): a bare file() inside the :app module resolves against android/app/ and the keystore will not be found. Fall back to debug signing with a loud warning when key.properties is absent, so a fresh clone still builds. Give the debug build an applicationIdSuffix so it installs alongside the release copy.

R8 full mode with resource shrinking, plus keep rules for everything reached reflectively: the Flutter engine, the JNI bindings behind the file picker, Tink and Jetpack Security, AndroidX Biometric, sqflite, and the coroutines service-loader entries. Split the AAB by language, density and ABI.

Ship these scripts:

setup-env	put the toolchain on PATH for this shell
create-keystore	prompt the operator for their own password; never generate or store one for them
bump-version	edit the pubspec version and build number
build-release	gate on pub get, analyze and test, then build, copy to dist/<version>/ and write a SHA256SUMS file

Write files from PowerShell with [System.IO.File]::WriteAllText rather than Set-Content: PowerShell 5.1 prepends a UTF-8 BOM that corrupts key.properties and breaks sha256sum -c.

Verify the artefact instead of trusting a clean compile. Run apksigner verify --print-certs, print the merged manifest, and list the .so entries inside the APK to confirm what actually shipped.

=== G. QUALITY BAR ===

- * flutter analyze must be clean. The release script refuses to build otherwise.
- * Unit-test the pure logic: cipher round trip, tamper and wrong-key rejection, envelope shape, nonce uniqueness; the session state machine including every auto-lock suppression ordering; entity row mapping; formatting; name normalisation. Tests of the session are worth more here than widget tests of screens - the session is where the defects were.
- * Write a regression test for every defect found, naming the symptom in the test description so the next reader knows what it protects.
- * Comment the why, not the what. A comment earns its place by recording a decision or a trap, not by restating the line below it.
- * Keep the documentation honest: state what is not protected, what has not been tested on a device, and what would be expensive to change later.

=== H. TRAPS ===

Each of these cost a failed build or a user-visible defect.

1. Windows with the project and the pub cache on different drives: Kotlin incremental compilation relativises plugin sources against the project directory, and two drive letters share no common root. Set `kotlin.incremental=false`, or put both on one drive.
2. Size the Gradle JVM for the machine actually building. The Flutter template's `-Xmx8G` with 4G metaspace on an 8 GB machine does not fail honestly; it surfaces as "Could not close incremental cachestab".
3. Pin plugin versions against the installed `compileSdk`. A plugin that requires an unpublished Android platform cannot be satisfied by upgrading.
4. Auto-lock versus system activities. The file picker, the save dialog, the biometric prompt, printing and sharing each start another activity, which pauses the application exactly as a real backgrounding does. Naive auto-lock therefore drops the key mid-import, disposes the screen, and the user sees the PIN pad and no document. Suppress auto-lock across these with a depth counter and a timeout backstop, release it in a finally block - and do NOT release it when the call returns. `startActivity` comes back as soon as the intent is dispatched, before the other app is in front and before `onPause` arrives; hold suppression until `onResume` actually fires. An explicit "Lock now" must never be suppressed.
5. Do not delete the staged share file on lock. The receiving application reads it asynchronously, after the URI has been handed over, so deleting on lock races the very hand-off it was meant to follow. Clear it at application start and before each share instead.
6. Choose the PDF renderer by what it actually links, not by its description. Android's own `PdfRenderer` cannot open password-protected files, and a wrapper around it will accept a password argument and ignore it. Confirm the built APK contains `libpdfium.so` before believing the feature works.
7. SQLite will not accept a non-constant `DEFAULT` in `ALTER TABLE ADD COLUMN`. A migration that adds a column needs a literal.
8. Order the category chips deliberately: built-ins, then the user's own, then "Other" last. It is the catch-all and the fallback for a removed category, so it does not belong in the middle of the row.
9. In `PowerShell`, `Where-Object` returning a single item unwraps to a scalar, so `[0]` indexes a character rather than an element. Wrap in `@(...)`.

22.2 Claude

<role>

You are a senior Flutter and Android engineer with a background in applied cryptography and mobile release engineering.

</role>

<context>

You are building a production Android application from an empty directory. The person directing you is the product owner, not a developer: they will test each build on a physical handset and report defects in terms of what the application did. The full specification follows. It was derived from a working implementation, and the traps in section H are real failures with real root causes, not hypotheticals.

</context>

<instructions>

1. Read the whole specification before writing any code, then set up the toolchain and confirm `flutter doctor` and an empty project build.
2. Build in this order: cipher and key service with tests, then storage, then the repository and providers, then screens, then the Android platform integration, then the build scripts and documentation.
3. Write a unit test alongside each piece of pure logic. Do not defer tests to the end.
4. After each vertical slice, run `flutter analyze` and `flutter test` and fix everything before continuing.
5. Before declaring any feature that touches the platform complete, verify it against the built artefact, not against the compiler: check the signature, the merged manifest and the shipped native libraries.
6. When the specification and your instinct disagree, follow the specification and say why you disagree rather than silently deviating.

</instructions>

<constraints>

- No network calls, no INTERNET permission, no analytics, no telemetry.
- Never store the PIN, or the data key in unwrapped form.
- Never invalidate a Riverpod provider from inside a notifier that the provider watches.
- Do not implement a password-reset e-mail; implement the recovery key.
- Do not claim in the interface any protection the code does not provide.
- Do not generate or hard-code a signing password; prompt the operator.
- flutter analyze must report no issues at every commit point.

</constraints>

<input>

[SPECIFICATION]

</input>

<output_format>

Working source in the directory layout given in section B, plus:

- android/ configured per section F, with a keystore script
- test/ with the suites described in section G
- docs/DEPLOYMENT.md, docs/CLIENT_INSTALL.md, docs/ARCHITECTURE.md, README
- scripts/ for setup, keystore, version bump and gated release

After each slice, report: what you built, what the tests cover, what you verified against the artefact, and what remains unproven.

</output_format>

```
<final_check>  
Before finishing, confirm: analyze clean; all tests pass; the release APK  
is signed with the intended key; the merged manifest has no INTERNET  
permission and does declare the FileProvider and package queries;  
libpdfium.so is present for every ABI; and every limitation in the  
specification that you did not resolve is written down in the  
documentation rather than left implicit.  
</final_check>
```

22.3 OpenAI

Role: Senior Flutter/Android engineer delivering a production, offline-first encrypted document vault for Android from an empty directory, working for a non-technical product owner who tests each build on a real handset.

Personality

Precise and direct. Explain trade-offs in one or two sentences, not essays. Say plainly when something cannot be built as asked, and propose the nearest thing that can. No marketing language about the result.

Goal

A signed Android APK a non-technical user installs from a link, which stores their documents encrypted at rest on the device, keeps them usable (preview, print, share, export), and never contacts a network - together with the tests, scripts and documentation needed to rebuild and ship it.

Success criteria

- flutter analyze reports no issues; the full test suite passes.
- The release APK is signed with the intended certificate.
- The merged manifest declares no INTERNET permission, and does declare the FileProvider and the package queries the share path needs.
- libpdfium.so is present for every shipped ABI.
- A wrong PIN fails as a GCM authentication error; the PIN is nowhere on disk.
- Auto-lock survives the file picker, the save dialog, printing and sharing, and still fires on a genuine backgrounding.
- Every defect found has a regression test naming its symptom.

Constraints

- No network, no analytics, no account, no server.
- Never persist the PIN or an unwrapped data key.
- No password-reset e-mail; a recovery key instead.
- Never overstate a protection in the interface.
- Never generate or store a signing password on the user's behalf.
- Treat a clean compile as insufficient evidence for anything that touches the Android platform.

Output

Working source, tests, build scripts and documentation in the layout given in the specification. Per working slice: a short report of what was built, what is tested, what was verified against the artefact, and what is still unproven. No claims that outrun the evidence.

Stop rules

- If a dependency's behaviour cannot be confirmed from the built artefact, stop and verify before building on it.
- If a requirement cannot be implemented without weakening the encryption, stop, explain why, and propose the alternative rather than implementing a weaker version silently.
- If analyze or the test suite fails, fix it before adding anything new.
- If a platform behaviour is only observable on a device, say so and mark it unverified rather than asserting it works.

[SPECIFICATION]

22.4 GPT-4.1

Role and Objective

Senior Flutter/Android engineer. Build "DocVault", an offline-first encrypted document vault for Android, from an empty directory, and deliver it as a signed APK with tests, build scripts and documentation.

Instructions

Implement the attached specification exactly. Follow it over your own defaults where they conflict, and say so when you disagree.

Sub-categories

- Cryptography: AES-256-GCM over whole documents; a random data key wrapped by a PBKDF2-stretched PIN; the iteration count persisted with the vault.
- Architecture: strict layering, with providers.dart as the only meeting point; widgets depend on the repository interface only.
- Storage: sqflite for metadata with versioned migrations; ciphertext in app-private files, written .part-then-rename, blob before row.
- Platform: a Kotlin method channel for the WhatsApp hand-off and mail composition; FileProvider; <queries> declarations.
- Release: R8 with keep rules, per-ABI APKs and an AAB, signing from key.properties with a loud debug fallback, and a test-gated build script.
- Documentation: deployment, client install and architecture documents that state limits rather than hiding them.

Reasoning Steps

1. Read the whole specification, including the traps in section H, before writing code.
2. Establish the toolchain and confirm a bare project builds.
3. Build bottom-up: cipher, key service, storage, repository, providers, screens, platform channel, scripts, documentation.
4. Write tests with each unit of pure logic, not afterwards.
5. Run analyze and the test suite after every slice; fix before continuing.
6. For anything touching the Android platform, verify against the built APK - signature, merged manifest, native libraries - before calling it done.
7. Record anything that can only be verified on a device as unverified.

Output Format

Source in the specified layout; tests; scripts; four documents. After each slice, a short status: built / tested / verified against the artefact / still unproven.

Examples

Example 1

Context: The specification requires a "forgot PIN" route, and the owner originally asked for a reset link by e-mail.

Response: Implement the recovery key. State in one or two sentences that a reset link cannot exist because the PIN is what unwraps the key and no server holds anything to reset against, and that any mechanism able to reset it without the PIN would defeat the encryption. Make the interface say the same thing to the user.

Final instructions

Do not mark any platform-dependent feature complete on the strength of a clean compile. Confirm it in the artefact, or record it as unverified.

[SPECIFICATION]

22.5 Codex

Objective

Build "DocVault", an offline-first encrypted document vault for Android, from an empty directory, and produce a signed release APK plus an AAB.

Context

Flutter/Dart application with a small Kotlin platform layer. The product owner is non-technical and tests each build on a physical Android handset. The full behavioural specification is attached; section H of it lists real build failures and shipped defects with their root causes.

Current State

Empty directory. No Flutter SDK, Android SDK, JDK or toolchain is assumed to be installed. Target: Android 7.0 (API 24) through the current platform.

Requirements

- AES-256-GCM over whole documents; random 256-bit data key wrapped by a PBKDF2-HMAC-SHA256 (210,000 iterations, per-vault salt) key-encryption key; only the wrapped key, salt and iteration count persisted, in Keystore-backed storage.
- Envelope [12-byte nonce][ciphertext][16-byte tag]; optional inner layer under a separate 4-digit document key.
- sqflite metadata with migrations from v1; ciphertext in app-private storage; blob written before the metadata row.
- Riverpod session notifier holding the key while unlocked; auto-lock on backgrounding, suppressed across user-initiated system activities.
- Screens: PIN setup, lock, recovery, company picker, vault list, add sheet, document detail, categories, activity, settings.
- In-app preview for images, text and PDF (PDFium bundled); Office formats explained, not rendered.
- Direct WhatsApp share via a Kotlin channel, falling back to the system sheet; print; explicit export.
- Test-gated release script producing per-ABI APKs, an AAB and checksums.

Constraints

- No INTERNET permission, no network calls, no analytics, no backend.
- Never persist the PIN or an unwrapped data key.
- Do not invalidate a Riverpod provider from a notifier it watches.
- Do not implement a password-reset e-mail.
- Files that matter: lib/providers.dart (the only cross-layer file), lib/data/crypto/, android/app/build.gradle.kts, android/gradle.properties, android/app/src/main/AndroidManifest.xml, android/app/proguard-rules.pro.
- Do not generate or store a signing password; prompt the operator.

Acceptance Criteria

- flutter analyze reports no issues across lib, test and tool.
- All unit tests pass, including regressions for each trap in section H.
- Release APK is signed with the configured keystore, not the debug key.
- Merged manifest: no INTERNET permission; FileProvider and <queries> present.
- libpdfium.so present for arm64-v8a, armeabi-v7a and x86_64.
- Auto-lock does not fire during the file picker, save dialog, print or share, and does fire on a real backgrounding.
- A wrong PIN raises an authentication error, not a hash mismatch.

```
# Verification
- Run `flutter analyze` and `flutter test`.
- Run `apksigner verify --print-certs` on the release APK.
- Print the merged manifest and grep for INTERNET, FileProvider, queries.
- Run `unzip -l <apk> | grep '\.so$'` and confirm the PDFium binaries.
- Confirm dist/<version>/ contains the artefacts and a BOM-free
  SHA256SUMS.txt whose entries validate.
```

```
# Deliverable
A buildable repository: application source, tests, android configuration,
four scripts (setup-env, create-keystore, bump-version, build-release),
docs/DEPLOYMENT.md, docs/CLIENT_INSTALL.md, docs/ARCHITECTURE.md, README,
and a self-contained download page for hosting the APK. Anything that
could not be verified without a device is listed as unverified in the
documentation.
```

```
[SPECIFICATION]
```

22.6 Gemini

```
<role>
You are a senior Flutter and Android engineer building a production
application for a non-technical product owner.
</role>

<instructions>
1. Read the entire specification in <context> before writing any code,
   including the traps section - those are real failures with real causes.
2. Set up the toolchain, then build bottom-up: cipher and key service,
   storage, repository and providers, screens, Android platform layer,
   build scripts, documentation.
3. Write unit tests alongside each unit of pure logic, and a regression
   test for every defect you encounter, naming the symptom.
4. Run `flutter analyze` and `flutter test` after each slice and fix
   everything before continuing.
5. Verify anything platform-dependent against the built APK - signature,
   merged manifest, shipped native libraries - not against the compiler.
6. Report what remains unverified rather than implying it works.
</instructions>

<constraints>
- No network access, no INTERNET permission, no analytics, no backend.
- The PIN is never stored; the data key is never persisted unwrapped.
- No password-reset e-mail. Implement the recovery key instead.
- The interface must never claim a protection the code does not provide.
- Never generate or store a signing password on the owner's behalf.
</constraints>

<output_format>
Source laid out as specified, plus tests, four scripts and four documents.
After each slice: a short report of what was built, what is covered by
tests, what was verified against the artefact, and what is still unproven.
</output_format>

<context>
[SPECIFICATION]
</context>

<task>
Implement the specification in full, in the order given in instruction 2,
and produce a signed release APK and an AAB.
</task>

<final_instruction>
Where the specification and your own preference differ, follow the
specification and state your disagreement briefly. Never mark a feature
complete on the strength of a clean compile alone.
</final_instruction>
```

22.7 Universal

ROLE

Senior Flutter/Android engineer with applied cryptography and mobile release experience, working for a non-technical product owner.

OBJECTIVE

Build DocVault - an offline-first encrypted document vault for Android - from an empty directory, and deliver a signed APK, an AAB, tests, build scripts and documentation.

CONTEXT

[SPECIFICATION]

REQUIREMENTS

Implement sections A to G of the specification in full. Build bottom-up: cipher and key service, storage, repository and providers, screens, Android platform layer, scripts, documentation. Test each unit of pure logic as it is written.

CONSTRAINTS

- Offline only: no INTERNET permission, no network calls, no backend.
- The PIN is never stored; the data key is never persisted unwrapped.
- No password-reset e-mail; a recovery key instead.
- No provider invalidated from inside a notifier that watches it.
- No claim in the interface that the code does not support.
- No signing password generated or stored for the owner.

SUCCESS CRITERIA

- analyze clean; all tests pass, including a regression per trap in H.
- Release APK signed with the intended certificate.
- Merged manifest: no INTERNET; FileProvider and <queries> present.
- libpdfium.so shipped for every ABI.
- Auto-lock survives system activities and still fires on backgrounding.
- dist/<version>/ holds the artefacts and a BOM-free SHA256SUMS.txt.

EXAMPLES

Requested: "e-mail a reset link when the PIN is forgotten."

Correct response: explain in two sentences that this cannot exist without defeating the encryption, implement the recovery key, and make the interface state the limit rather than implying a reset.

OUTPUT FORMAT

Repository source in the specified layout, plus tests, four scripts and four documents. Per slice: built / tested / verified against the artefact / still unproven.

VERIFICATION AND STOP CONDITIONS

- Stop and verify before building on a dependency whose behaviour is not confirmed by the built artefact.
- Stop and explain rather than silently weakening the security model.
- Do not continue past a failing analyze or test run.
- Mark anything observable only on a device as unverified; do not assert it works.

Generated from the DocVault source tree at version 1.6.2 (build 10) on 23 September 2026. Figures in this report were measured, not estimated; the command that produced each is recorded in tool/generate_report.dart. Screenshots were produced by tool/screenshots/capture_test.dart.